Physics-Constrained Repair of Quantum Circuits: An Audited Benchmark, a Leakage-Free Protocol, and Controlled Ablations

Alok Pandey

Abstract—Success rates reported for automated quantum-circuit repair are only meaningful if the repair procedure cannot read the reference it is scored against and the benchmark itself is valid. We audit the first release of this physics-constrained repair framework and its benchmark and find both conditions violated: the reference circuit was returned as the repair, baseline outcomes were hard-coded, 367 of 1000 reference circuits failed the validator's own connectivity check, and the semantic check was blind to controlled-phase faults in the QFT. We specify a protocol that fixes these defects (a query-counted behavioural oracle as the only information channel, an exact-unitary and noise-floor acceptance predicate, deterministic verdicts), release a corrected 1000-circuit benchmark whose references are hardware-admissible and whose faults are observable by construction, and evaluate a repair agent (analytic rotation-angle solving, oracle-guided edit search, counterexample-guided feedback) in a 2×2 design that separates physical knowledge from feedback. At a matched attempt budget of three, the full agent is accepted on 97.0% of circuits (95% CI $[95.7, 97.9]$) against 77.0% without physical knowledge, 60.0% for static repair without an oracle, 29.7% for the Qiskit transpiler alone and 5.0% for ZX-calculus simplification. The physical-knowledge effect (+20.0 points, 200 discordant circuits, none reversed) is confined to the physical fault class, where an arm without routing cannot succeed; on logical faults the arms are indistinguishable. Diagnostic feedback has no effect when the oracle is strong but raises Pass@3 from 88.1% to 97.0% (+8.9 points, $p < 10^{-24}$) when the initial test suite is a single state. On 600 programs with measurement, reset, classical control and loops, guided repair fixes 92.3% against 48.5% for random search, and a calibration-based estimate shows that the benchmark's identity layout is much looser than a compiler layout (median success estimate 0.66 against 0.89). As the only repair engine on 60 circuits, a 1.5-billion-parameter local model repairs 16.7%, two small Gemini models 43.3% and 51.7%, and a larger one, gemini-3.5-flash, 86.7%, against 98.3% for guided search; the last gap is not significant after correction ($n = 60$), and these are single runs of four models, not verdicts on language models. When the two small models drive oracle-scored repair tools instead of writing the circuit they reach 90–95%, but the tools alone, with no model, reach 98.3%. A guard added after reading the benchmark's failures lifts the extended agent from 98.8% to 99.7%, a development-set number. On five real Bugs4Q bugs guided search repairs three. No circuit was run on real hardware.

Index Terms—Quantum software engineering, automated program repair, circuit debugging, hardware-aware compilation, benchmark validity, evaluation methodology.

The author is with the Indian Institute of Technology Patna, India.

Code, data, per-attempt results and the analysis scripts required to reproduce every number in this paper are available at https://github.com/alok-108/physics-constrained-quantum-repair.

I. Introduction

A quantum program must satisfy two conditions that are easy to conflate: it must compute what the specification demands, and it must be executable on the target device [1]. A repair that fixes a parser error can introduce a wrong operand; a semantically correct abstract circuit can still violate connectivity or exceed a coherence budget. Evaluating a repair system therefore needs an acceptance predicate covering both, and a benchmark in which that predicate can actually be satisfied by a correct repair.

Recent work studies language-model-assisted repair through direct patch generation [2] and through agentic pipelines with bug injection, detection, repair and simulation-based validation [3], alongside symbolic and learned circuit transformation [4], [5], [6]. The common evaluation pattern is to compare a candidate with a reference circuit and report agreement as success. This is sound only if (i) the repair procedure could not have consulted the reference, (ii) the reference is itself acceptable under the predicate, and (iii) every injected fault is detectable by that predicate. In a multi-component agent the reference tends to be reachable somewhere in the call graph, and benchmarks generated by script are rarely checked against their own validator.

We examined the first release of this repository's physics-constrained repair framework and benchmark for exactly these properties and report what we found (Section III). The results it originally reported (Pass@1 77.5%, Pass@3 99.1%) cannot be used. This paper makes the following contributions.

1) An audit of the original harness and benchmark, with a reusable tool (src/audit.py) that checks reference admissibility and fault observability on any dataset.
2) A leakage-free protocol: an explicit information boundary, a query-counted behavioural oracle, a deterministic acceptance predicate with an exact noise-floor criterion, and a held-out overfitting measurement.
3) Benchmark v2: 1000 circuits with hardware-admissible references and verified-observable faults, labelled by fault mode so results can be read per mode rather than only per class.
4) A repair agent using analytic rotation-angle solving, meet-in-the-middle scoring of whole edit neighbour-

hoods, tabu diversification, and counterexample-guided feedback.

5) Controlled experiments: a 2×2 factorial design (physical knowledge $\times$ feedback) with a no-oracle lower bound and a transformation baseline, a weak-oracle regime, and a query-budget sweep.

We state limits up front. The benchmark consists of small edits of short circuits whose inverse lies inside the agent's edit neighbourhood, so a high rate on single-edit faults is expected and is not evidence of general repair ability; the informative cells are the fault modes outside the neighbourhood and the weak-oracle and low-budget regimes. Hardware is a device model, connectivity is checked under the identity layout, and the language models we ran are small ones (a local 1.5-billion-parameter model and two small Gemini models).

II. Related Work and Scope

Language-guided repair. Guo et al. [2] apply ChatGPT to repair quantum programs. QBugLM [3] provides an agentic benchmarking framework and reports that iterative feedback matters. Any comparison with such a system must match retry budgets and run the authors' release; we therefore report no arm named after an external system. Published quantum repair methods with their own protocols include HornBro [7] and the unitary-operation approach of Li et al. [8]; they differ from ours in task, language and oracle, so we do not claim a head-to-head comparison with them. HornBro's code is public, and a comparison on our benchmark needs its authors' pipeline run under our predicate (Section X).

Benchmarks. Bugs4Q [9] remains the main collection of real, executable quantum bugs with reproducing tests, but it is small and tied to Qiskit, and its entries are sensitive to SDK versions; reproducibility drops without version pinning [10]. Our benchmark is synthetic and therefore complementary: it trades realism for control over admissibility and observability, and it makes no claim about real-fault prevalence. Surveys of quantum testing identify the oracle problem as central and stress methodological standards [11]; measurement-based oracles that use only computational-basis outputs can miss phase faults [12], which is the failure we found in the single-input check.

Debugging tools. MQT Debugger [13] evaluates user-placed assertions by classical simulation and, after a failure, narrows the cause by cone-of-influence analysis; Proq [14] checks projection-based assertions with few measurements, including on hardware. Both detect and locate faults but need hand-written assertions and leave the repair to the user. The tool released with this paper (qrepair) takes a reference circuit or amplitude tests instead, localises by scoring edits, and proposes a minimal diff; we make no head-to-head claim (docs/COMPARISON.md).

Transformation baselines. ZX-calculus simplification [4] rewrites a circuit while preserving the computation it denotes, which makes PyZX a meaningful transformation baseline but not a repair baseline: it cannot restore a computation the faulty circuit does not implement. Quarl [6] learns a rewriting policy and meets the same boundary.

Test-suite-guided repair. In classical program repair the tool receives a test suite, not the fixed program [15], and overfitting to that suite is a known hazard [16]. Counterexample-guided inductive synthesis [17] adds failing inputs to the suite. Our oracle and feedback arms transplant both ideas to circuits.

Parameter optimisation. For a circuit with one free rotation angle, a measured quantity is a low-degree trigonometric polynomial of that angle, so its optimum can be located analytically from a few evaluations; this underlies sequential minimal optimisation [18] and structure optimisation [19] for variational circuits. We use the same property to repair a wrong angle in three oracle queries.

Execution models. Hardware checks use the FakeSherbrooke backend model; backend-derived noise models approximate device behaviour [20], so passing them is distinct from fidelity on a live processor.

TABLE I
Audit of the original (v1) harness and benchmark. Counts are over its 1000 circuits unless stated.

Finding	Evidence
Reference circuit returned as the repair (ground_truth_hint)	code
Baseline outcome hard-coded to failure for all physics-fault circuits	code
Baseline Pass@3 set equal to Pass@1 (one attempt vs. three)	code
Arm named "QBugLM" was the project's own repairer	code
Injected fault label passed to the diagnostician	code
Reference circuits failing the validator's connectivity check	367 (250 QFT, 117 QAOA)
Structural/semantic faults that leave the unitary unchanged	20 of 400
Single-input semantic check blind to QFT phase and swap faults	fidelity 1.0000
No acceptance stage separating a redundant circuit from its reference	by construction

III. Audit of the Original Benchmark

We audited the original release against its own validator. Table I summarises the findings; per-defect locations and fixes are in docs/BENCHMARK_V2.md.

The connectivity and blindness findings are not harness bugs but properties of the benchmark and predicate, and they survive fixing the leakage. First, a candidate can be semantically perfect and still be rejected, and any system lacking a routing step cannot pass those circuits; the apparent value of "physical awareness" then depends on circuit family rather than on the injected fault. Second, a QFT acting on $|0 \ldots 0\rangle$ yields the uniform superposition, so a circuit consisting only of the Hadamard layer and a circuit with every controlled phase negated

both have overlap 1.0000 with the reference (checked on real references); such faults are unobservable under a single-input check.

IV. Evaluation Protocol

A. Information boundary

Let $\mathcal{I}_R$ be the information available to the repair procedure and $\mathcal{I}_V$ that available to the acceptance predicate. A valid evaluation requires

$$C_{\text{ref}} \in \mathcal{I}_V, \quad C_{\text{ref}} \notin \mathcal{I}_R, \quad \beta \notin \mathcal{I}_R, \tag{1}$$

where C_{ref} is the reference and β the injected fault class and mode. If $C_{\text{ref}} \in \mathcal{I}_R$ the trivial procedure that emits it is accepted on every circuit; if $\beta \in \mathcal{I}_R$ localisation is handed its answer. We enforce (1) in code: the benchmark's annotation fields are stripped from the specification before any repair-side component sees it, the repair interface has no reference argument, and unit tests check both.

B. Behavioural oracle

The reference is wrapped in an oracle that exposes only a scalar agreement score and a query counter. It privately holds three disjoint sets of input states: a suite guiding the search ($|0\dots0\rangle$ plus Haar-random product states), a pool of candidate counterexamples, and a held-out set never used for search or feedback. The score of a candidate C is the mean agreement $\Omega(C) = \frac{1}{m}\sum_j |\langle \psi_j^{\text{ref}} \mid \psi_j^C \rangle|^2$ over the current suite. Every scored edit, score call and counterexample request consumes one query of a per-repair budget (default 8000). The oracle's search signal is deliberately weaker than the acceptance predicate: the latter checks the full unitary, which makes overfitting measurable as the gap between suite agreement and exact process fidelity of the final candidate.

C. Acceptance predicate

Acceptance is a conjunction, evaluated from inexpensive to expensive stages and identical for every system:

$$\mathcal{A}(C; R, B) = V_{\text{syn}} \wedge V_{\text{str}} \wedge V_{\text{phys}} \wedge V_{\text{min}} \wedge V_{\text{sem}} \wedge V_{\text{noise}}. \tag{2}$$

V_{syn} requires that the candidate parse as OpenQASM 3; V_{str} that it declare the specified register width; V_{phys} that every two-qubit gate act on adjacent qubits of FakeSherbrooke (identity layout), that per-qubit execution time stay under half of $\min(T_1, T_2)$, and that depth not exceed 250; V_{min} that no adjacent self-inverse pair remain, which is needed because such pairs preserve the unitary and an untouched redundant circuit would otherwise be accepted; and

$$V_{\text{sem}}: \quad \frac{|\text{tr}(U_C^\dagger U_R)|^2}{d^2} \geq 0.99, \tag{3}$$

exact unitary agreement up to global phase, which replaces the single-input overlap of the original benchmark.

D. Exact noise-floor criterion

A fixed total-variation threshold is not comparable across circuits, and a sampled test is stochastic: a correct repair identical to the reference can be rejected by shot noise, and the rejection cannot be undone by resubmission under deterministic seeding. We instead compute the exact noisy output distributions p_C, p_R by density-matrix simulation under a depolarising model (single-qubit 2.5×10^{-4}, two-qubit 7.5×10^{-3}) with symmetric readout error 1.5% applied analytically, and accept when

$$\text{TVD}(p_C, p_R) \leq \tau_{0.99}, \tag{4}$$

where $\tau_{0.99}$ is the 99th percentile of the TVD between two independent 4096-shot samples of p_R (400 resamples). Acceptance therefore means that the difference lies below the sampling noise floor of a 4096-shot experiment. The verdict is deterministic, and a candidate identical to the reference has TVD 0 and is never falsely rejected. A debugging run during development exposed exactly this failure of the sampled version: a repair with process fidelity 1.0 was rejected at TVD 0.0500 against a sampled threshold of 0.0467.

V. Benchmark v2

The generator is deterministic (seed 42) and emits 1000 circuits: 250 per family (GHZ, QAOA MaxCut, VQE H_2, QFT; 3–8 qubits), 200 per fault class. It enforces two invariants and re-samples any draw that violates them; the audit tool reports 0/1000 violations of either.

Admissible references. Device-agnostic circuits that need routing (QFT, QAOA with a ring edge) are routed with swap-in/swap-back and inverse cancellation, which preserves the unitary exactly, so routed and unrouted circuits are compared without layout bookkeeping. Every reference is admissible and irreducible.

Observable faults. Syntax faults do not parse. Structural and semantic faults lower process fidelity below 0.99 while keeping the circuit admissible. Redundancy faults are removable by inverse cancellation, which restores the reference. Physical faults fail V_{phys} while being semantically correct: either a device-agnostic circuit that needs routing (reference: its routed form) or a 1200 μs idle delay (reference: without it).

Each record carries a fault_mode and a mode_family. "Legacy" modes are those of the original injector (angle negation and $\pi/2$ shift, axis change, gate replacement, gate removal, reversed CNOT, wrong qubit, redundant pairs, four syntax corruptions). "Extended" modes were added here: angle scaling by 1.5, $\pi/3$ shift, simultaneous two-fault ("compound") semantic faults, and CX→CZ replacement.

VI. Repair Agent

A. Loop and arms

A bounded loop of at most $K = 3$ attempts runs diagnosis, candidate generation and acceptance, and reports budget exhaustion as unresolved rather than as acceptance of the last patch (Fig. 1). Two independent

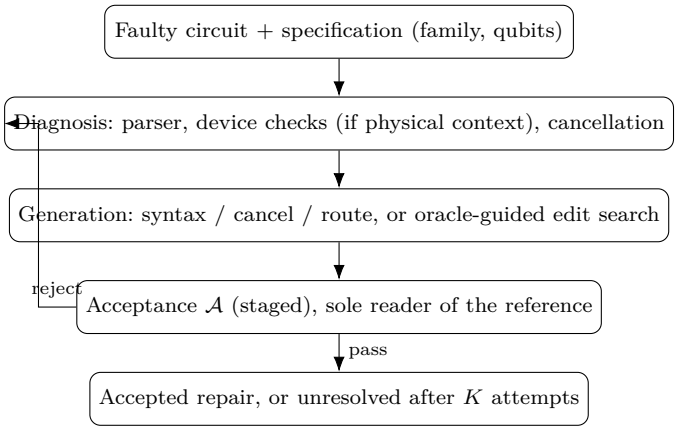


Fig. 1. Bounded repair loop. The reference reaches only the acceptance predicate and, wrapped as a query-counted oracle, the generation stage. Diagnostic feedback carries the rejecting stage and, on a semantic rejection, a counterexample test.

switches define the ablation: physical context gates the device checks in diagnosis and the delay trimming and routing in generation; diagnostic feedback decides whether a rejection returns its stage and, for semantic rejections, a counterexample test, or only a bare rejection. The acceptance predicate is never ablated.

B. Deterministic repairs

Syntax faults are repaired by token-level recovery (keyword and semicolon restoration, restoring a lost closing bracket after its index). Redundancy is repaired by DAG inverse cancellation. A physical fault is repaired, when physical context is on, by dropping idle delays beyond $150\,\mu$s and then routing with swap-in/swap-back followed by cancellation. The router is the same construction used to build physical-fault references; routing agreement is therefore partly by construction (Section X).

C. Oracle-guided edit search

Structural and semantic faults are repaired by greedy hill climbing over single-instruction edits, each costing one oracle query. The neighbourhood is generic and not conditioned on a fault label: analytic angle moves (below); substitution of a parameter-free single-qubit gate among $\{H, X, Y, Z, S, S^\dagger, T, T^\dagger\}$; rotation-axis exchange; reversal of CX operands; re-targeting a single-qubit gate to another qubit; deletion of an instruction; and insertion of H, X, adjacent CX or adjacent SWAP at any position. Not in the neighbourhood, by design: two-qubit gate substitution, insertion of rotations, and simultaneous multi-instruction edits beyond the greedy depth of four steps.

Analytic angle solving. For a gate $e^{-i\theta G/2}$ with $G^2 = I$ (single-qubit rotations) or a controlled phase $\mathrm{diag}(1, \ldots, e^{i\theta})$, the amplitude of any fixed input–output pair is $ae^{-i\theta/2} + be^{i\theta/2}$ or $a + be^{i\theta}$, so its squared modulus, and hence the suite score, is

$$f(\theta) = c_0 + c_1 \cos\theta + s_1 \sin\theta. \tag{5}$$

TABLE II
Systems. P/U: physical context on/off; D/R: diagnostic feedback or bare rejection.

Code	Description	Phys.	Feedback	Oracle
P+D	full agent	on	diag.	yes
P+R	bare rejection	on	bare	yes
U+D	no physical context	off	diag.	yes
U+R	no phys., bare rej.	off	bare	yes
Static	static repairs only	on	–	no
PyZX	ZX simplification	–	–	no
P+D/Q	P+D, Qiskit router	on	diag.	yes
Qiskit	compiler only, O0–O3	–	–	no

Three probes at $\theta \in \{0, 2\pi/3, 4\pi/3\}$ determine (c_0, c_1, s_1) by a well-conditioned 3×3 solve, and the maximiser is $\theta^\star = \mathrm{atan2}(s_1, c_1)$ with predicted score $c_0 + \sqrt{c_1^2 + s_1^2}$. A wrong angle is thus corrected in three queries per rotation instruction, whatever the perturbation.

Meet-in-the-middle scoring. The score of every single-instruction edit of a base circuit is computed from cached forward states (prefixes applied to the suite inputs) and backward states (suffixes inverted and applied to the reference outputs), so an edit costs one local gate application rather than a full simulation. Values equal naive scoring to 10^{-10} (unit-tested); budget accounting still charges one query per edit, and only scalars leave the oracle.

Tabu diversification. Submitted circuits enter a tabu set, so a retry returns a different candidate and attempts are not spent on byte-identical resubmissions.

D. Counterexample-guided feedback

When $\mathcal{A}$ rejects a candidate at the semantic stage and diagnostic feedback is on, the oracle moves the pool state on which the candidate disagrees most into the suite and the search restarts from the candidate with the enlarged suite. This is counterexample-guided repair [17]. It makes the feedback ablation testable without a language model, because the feedback now changes what the generator can know.

VII. Experimental Design

Arms. Table II lists the systems (Qiskit is also run at levels 0, 2 and 3). All run through the same loop with the same $K = 3$ and the same acceptance predicate. No system is given the reference or the fault label.

Experiments. E1 (main): suite of 4 states, budget 8000 queries, the six core arms plus the compiler and router arms (run afterwards on the same circuits), all 1000 circuits. E2 (weak oracle): suite of 1 state ($|0 \ldots 0\rangle$), same budget, the four factorial arms. E3 (budget sweep): suite of 4, budgets $50, \ldots, 3200$, arms P+D and U+R.

Statistics. Pass@k is the fraction accepted within k attempts, read from the attempt trace. Rates carry Wilson 95% intervals. Outcomes are paired binary variables, so contrasts use McNemar's exact test [21] on discordant pairs with the paired risk difference and a Newcombe method-10 interval [22]; Holm correction [23] is applied

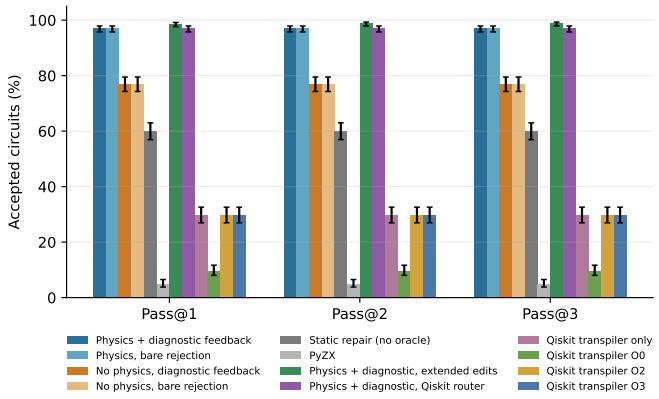


Fig. 2. Pass@k in E1 with Wilson 95% intervals. Attempts beyond the first add nothing in this regime.

over the rows of each contrast table. Six contrasts were fixed before running; the compiler and router contrasts were added after the first results and are labelled as such here. Resource overhead is gate count relative to the reference over accepted repairs only. All runs are deterministic.

VIII. Results

A. E1: main configuration

Table III and Fig. 2 give the primary outcomes. P+D is accepted on 97.0% of circuits within three attempts (95% CI $[95.7, 97.9]$) and every final candidate is physically valid. Without physical context the rate is 77.0% ($[74.3, 79.5]$) and 80.1% of final candidates are valid. Static repair without an oracle reaches 60.0% and ZX simplification 5.0%. Accepted repairs of the oracle arms are within 0.2% of the reference in gate count on average; PyZX output is 156.8% larger.

Table IV gives the contrasts. The physical-knowledge effect is $+0.200$ ($[0.176, 0.226]$; 200 circuits repaired by the physical arm and none in the reverse direction) and is the same with feedback on and off. The oracle effect is $+0.370$ ($[0.340, 0.400]$). Diagnostic feedback has no effect: 0 discordant pairs in both comparisons.

The physical-knowledge effect is a capability effect confined to the physical class. Table V shows 100% against 0% on the 200 physical-fault circuits and identical rates on the other four classes. An arm without delay trimming and routing cannot repair a routing or coherence fault, so this contrast measures the availability of those operations, which we report openly, rather than any improvement in semantic reasoning, on which the arms are indistinguishable. In the unconstrained arms 199 of the 200 physical faults are rejected at the connectivity (96) or coherence (103) stage (Table VI); one is rejected at the semantic stage.

B. Fault modes and the limits of the neighbourhood

Table VII reports Pass@3 per fault mode. All semantic modes, including the extended ones (scale, $\pi/3$ shift, compound), reach 100%: angle solving corrects any single wrong angle, and the greedy path resolves two simultaneous faults. The unresolved circuits (30, all in the structural class) are concentrated where the neighbourhood is incomplete. Of the 11 circuits whose removed instruction was a rotation or controlled phase, none is repaired, because rotations are not insertable; removal of a CX, H or X is repaired in all 42 cases and of a SWAP in 90% of 21. Retargeting a single-qubit gate is repaired in 97% of cases for H but only 50% for R_z (12 circuits). CX→CZ replacement, absent from the neighbourhood, is repaired in 63% of 27 circuits (GHZ 89%, QAOA 56%, VQE 44%), which requires a two-edit path. These cells, not the aggregate, indicate what the agent cannot do.

C. Compiler baseline and router robustness

A standard compiler is the natural baseline for the physical class. The Qiskit arm runs the transpiler (trivial layout, SABRE routing onto the device line, level-1 optimisation) and appends an exact permutation that undoes the final layout, so its output implements the same unitary as its input and is scored by the same predicate; it contains no repair logic. It is accepted on 29.7% of circuits ($[26.9, 32.6]$): 100% of redundancy faults (level-1 cancellation), 48.5% of physical faults (all 97 unmapped circuits, none of the 103 over-long delays, which the transpiler leaves in place), and none of the syntax, structural or semantic faults. P+D exceeds it by $+0.673$ ($[0.643, 0.701]$; 673 discordant circuits, none reversed), so compilation alone covers less than a third of the benchmark even in its home class. Replacing the agent's router by Qiskit's (P+D/Q) leaves acceptance unchanged (zero discordant circuits). Across Qiskit optimisation levels the compiler-only rate is 9.7% at level 0 (routing only: the 97 unmapped circuits), 29.7% at level 1 and likewise at levels 2 and 3, which add inverse cancellation; higher optimisation does not erase the benchmark's faults, because none except redundancy can be removed by compilation.

D. Extended edit neighbourhood and localisation

The per-mode table shows two gaps in the base neighbourhood. We therefore added, after E1, two operators behind a flag: substitution of a two-qubit gate (CX↔CZ, CX→CY) and insertion of a rotation or adjacent controlled phase with its angle solved from three probe queries, tried only when no single edit completes the repair. The extended agent (P+D/E) reaches 98.8% Pass@3 ($[97.9, 99.3]$): 20 circuits are repaired that P+D misses and 2 are lost, a paired difference of $+0.018$ ($[0.009, 0.029]$; Holm $p < 10^{-3}$; this contrast was added after the first results). CX→CZ replacement rises from 63% to 100% and removal of an instruction from 82% to 96%. Both lost circuits are compound QFT faults whose search exhausted the 8000-query budget (the base agent used 6817 and 1522 queries on them), so the extension costs budget on the hardest cases; mean queries are 546.5 against 552.9.

TABLE III
Acceptance under the shared predicate, $n = 1000$ circuits, attempt budget $K = 3$. CI: Wilson 95%. P/U = physics context on/off; D/R = diagnostic feedback / bare rejection; Static = no oracle; P+D/E = P+D with the extended edit neighbourhood; P+D/Q = P+D with Qiskit routing; Qiskit On = transpiler only at optimisation level n.

System	P@1 (%)	P@3 (%)	P@3 95% CI	Phys. valid (%)	Attempts	Queries
P+D	97.0	97.0	[95.7, 97.9]	100.0	1.06	553
P+R	97.0	97.0	[95.7, 97.9]	100.0	1.06	555
U+D	77.0	77.0	[74.3, 79.5]	80.1	1.46	963
U+R	77.0	77.0	[74.3, 79.5]	80.1	1.46	965
Static	60.0	60.0	[56.9, 63.0]	100.0	1.80	0
PyZX	5.0	5.0	[3.8, 6.5]	15.1	2.90	0
P+D/E	98.6	98.8	[97.9, 99.3]	100.0	1.03	546
P+D/Q	97.0	97.0	[95.7, 97.9]	100.0	1.06	553
Qiskit O1	29.7	29.7	[26.9, 32.6]	69.7	2.41	0
Qiskit O0	9.7	9.7	[8.0, 11.7]	69.7	2.81	0
Qiskit O2	29.7	29.7	[26.9, 32.6]	69.7	2.41	0
Qiskit O3	29.7	29.7	[26.9, 32.6]	69.7	2.41	0

TABLE IV
Pre-specified paired contrasts (exact McNemar; Newcombe interval on the paired risk difference; Holm correction over the 18 rows shown).

contrast	metric	b10	b01	RD [95% CI]	p_holm
physics context (feedback on)	Pass@1	200	0	+0.200 [+0.176, +0.226]	1.5×10^{-59}
physics context (feedback on)	Pass@3	200	0	+0.200 [+0.176, +0.226]	1.5×10^{-59}
physics context (feedback off)	Pass@1	200	0	+0.200 [+0.176, +0.226]	1.5×10^{-59}
physics context (feedback off)	Pass@3	200	0	+0.200 [+0.176, +0.226]	1.5×10^{-59}
diagnostic feedback (physics on)	Pass@1	0	0	+0.000 [-0.004, +0.004]	1
diagnostic feedback (physics on)	Pass@3	0	0	+0.000 [-0.004, +0.004]	1
diagnostic feedback (physics off)	Pass@1	0	0	+0.000 [-0.002, +0.002]	1
diagnostic feedback (physics off)	Pass@3	0	0	+0.000 [-0.002, +0.002]	1
behavioural oracle	Pass@1	370	0	+0.370 [+0.340, +0.400]	1.2×10^{-110}
behavioural oracle	Pass@3	370	0	+0.370 [+0.340, +0.400]	1.2×10^{-110}
transformation baseline	Pass@1	920	0	+0.920 [+0.901, +0.935]	4.1×10^{-276}
transformation baseline	Pass@3	920	0	+0.920 [+0.901, +0.935]	4.1×10^{-276}
compiler baseline	Pass@1	673	0	+0.673 [+0.643, +0.701]	8.2×10^{-202}
compiler baseline	Pass@3	673	0	+0.673 [+0.643, +0.701]	8.2×10^{-202}
extended edit neighbourhood (added after E1)	Pass@1	18	2	+0.016 [+0.008, +0.026]	0.00282
extended edit neighbourhood (added after E1)	Pass@3	20	2	+0.018 [+0.009, +0.029]	9.7×10^{-4}
router choice (own vs Qiskit)	Pass@1	0	0	+0.000 [-0.004, +0.004]	1
router choice (own vs Qiskit)	Pass@3	0	0	+0.000 [-0.004, +0.004]	1

TABLE V
Pass@3 (%) by injected fault class (200 circuits per class). P/U = physics context on/off; D/R = diagnostic feedback / bare rejection; Static = no oracle; P+D/E = P+D with the extended edit neighbourhood; P+D/Q = P+D with Qiskit routing; Qiskit On = transpiler only at optimisation level n.

bug_type	P+D	P+R	U+D	U+R	Static	PyZX	P+D/E	P+D/Q	Qiskit O1	Qiskit O0	Qiskit O2	Qiskit O3
gate_redundancy	100.0	100.0	100.0	100.0	100.0	25.0	100.0	100.0	100.0	0.0	100.0	100.0
physics_violation	100.0	100.0	0.0	0.0	100.0	0.0	100.0	100.0	48.5	48.5	48.5	48.5
semantic_deviation	100.0	100.0	100.0	100.0	0.0	0.0	99.0	100.0	0.0	0.0	0.0	0.0
structural	85.0	85.0	85.0	85.0	0.0	0.0	95.0	85.0	0.0	0.0	0.0	0.0
syntax	100.0	100.0	100.0	100.0	100.0	0.0	100.0	100.0	0.0	0.0	0.0	0.0

The same scoring gives instruction-level fault localisation: rank each instruction by the largest agreement gain of any single edit there. On the 400 structural and semantic faults, with the faulty position taken from the difference to the reference, the faulty instruction is ranked first in 86.5% of cases with the base neighbourhood and 86.2% with the extended one, within the top three in 91.0% and 93.8%, and within the top five in 92.8% and 95.2%, against 7.7%, 22.9% and 36.3% for a random ranking. Removed instructions are the weakest case (66.2% top-1, extended).

E. E2: weak oracle

With a suite of one state the first-attempt rate falls from 97.0% to 86.9% and feedback becomes decisive (Table VIII). P+D rises to 96.8% at the second attempt and 97.0% at the third; P+R reaches only 88.1% ($[85.9, 90.0]$). The feedback effect is +0.089 ($[0.072, 0.108]$; 90 circuits repaired with feedback and 1 without; Holm $p = 3.0 \times 10^{-25}$) and is identical with physical context on and off (Table IX). It is concentrated in the semantic class (98.5% against 70.5%) and the structural class (86.5% against 70.0%). A single state cannot distinguish

TABLE VI
Rejecting stage of the final attempt among unresolved circuits. P/U = physics context on/off; D/R = diagnostic feedback / bare rejection; Static = no oracle; P+D/E = P+D with the extended edit neighbourhood; P+D/Q = P+D with Qiskit routing; Qiskit On = transpiler only at optimisation level n.

final_failed_stage	P+D	P+R	U+D	U+R	Static	PyZX	P+D/E	P+D/Q	Qiskit O1	Qiskit O0	Qiskit O2	Qiskit O3
coherence	0	0	103	103	0	0	0	0	103	103	103	103
connectivity	0	0	96	96	0	264	0	0	0	0	0	0
generation	0	0	0	0	0	585	0	0	200	200	200	200
minimality	0	0	0	0	0	25	0	0	0	212	0	0
semantic	30	30	31	31	400	76	12	30	400	388	400	400

TABLE VII
Pass@3 (%) by fault mode. "legacy" modes come from the v1 injector; "extended" modes were added in v2. P/U = physics context on/off; D/R = diagnostic feedback / bare rejection; Static = no oracle; P+D/E = P+D with the extended edit neighbourhood; P+D/Q = P+D with Qiskit routing; Qiskit On = transpiler only at optimisation level n.

bug_type	fault_mode	mode_family	P+D	P+D/E	U+R	n
gate_redundancy	cxcx	legacy	100.0	100.0	100.0	64
gate_redundancy	hh	legacy	100.0	100.0	100.0	61
gate_redundancy	xx	legacy	100.0	100.0	100.0	75
physics_violation	coherence_delay	legacy	100.0	100.0	0.0	103
physics_violation	unmapped	legacy	100.0	100.0	0.0	97
semantic_deviation	axis_change	legacy	100.0	100.0	100.0	14
semantic_deviation	compound	extended	100.0	93.9	100.0	33
semantic_deviation	gate_replace	legacy	100.0	100.0	100.0	50
semantic_deviation	negate	legacy	100.0	100.0	100.0	27
semantic_deviation	scale	extended	100.0	100.0	100.0	16
semantic_deviation	shift_pi2	legacy	100.0	100.0	100.0	30
semantic_deviation	shift_pi3	extended	100.0	100.0	100.0	30
structural	invert_cnot	legacy	100.0	100.0	100.0	44
structural	remove_gate	legacy	82.4	95.9	82.4	74
structural	twoqubit_replace	extended	63.0	100.0	63.0	27
structural	wrong_qubit	legacy	87.3	87.3	87.3	55
syntax	corrupt_keyword	legacy	100.0	100.0	100.0	60
syntax	illegal_bracket	legacy	100.0	100.0	100.0	50
syntax	invalid_gate	legacy	100.0	100.0	100.0	37
syntax	missing_semicolon	legacy	100.0	100.0	100.0	53

a circuit that lacks a rotation from one that has it; a counterexample supplies the distinguishing input.

F. E3: query budget

Table X and Fig. 3 vary the per-repair oracle budget with a suite of four states. Pass@3 of P+D rises from 77.7% at 50 queries to 85.1% (100), 88.3% (200), 91.4% (400), 93.7% (800), 95.6% (1600), 96.7% (3200) and 97.0% at the default 8000, with diminishing returns: +7.4 points from 50 to 100 queries, +1.1 from 1600 to 3200 and +0.3 from 3200 to 8000. The 600 syntax, redundancy and physical-fault circuits that static repair resolves need no oracle and are accepted at any budget, so the budget governs only the 400 structural and semantic circuits: 44% of them are repaired within 50 queries, 92% within 3200. U+R trails P+D by exactly 20.0 points at every budget, the share of physical faults; budget and physical context do not interact.

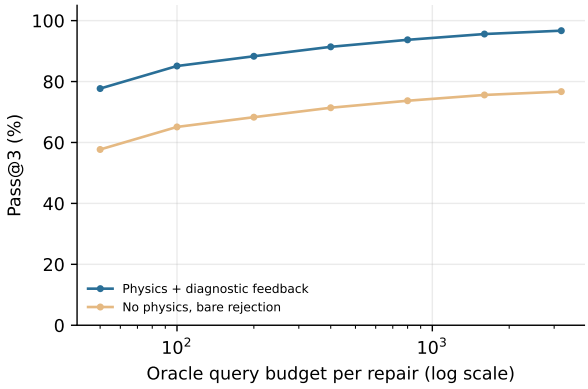


Fig. 3. Pass@3 as a function of the per-repair oracle query budget (E3).

G. Oracle cost and overfitting

Table XI reports cost. The median searched repair in P+D uses 508 queries, the 95th percentile 2347, and counterexamples are rare (0.066 per run). Accepted repairs cannot exhibit a suite–process gap above 0.01 by construction, so the gap is informative only among unresolved circuits: for 23 of the 30 unresolved final candidates of P+D, suite agreement exceeds exact process fidelity by more than 0.01, i.e. the candidate fits the test suite better than it fits the specification.

TABLE VIII
Weak-oracle regime (initial suite = $|0\ldots0\rangle$ only; oracle budget unchanged). P/U = physics context on/off; D/R = diagnostic feedback / bare rejection; Static = no oracle; P+D/E = P+D with the extended edit neighbourhood; P+D/Q = P+D with Qiskit routing; Qiskit On = transpiler only at optimisation level n.

System	P@1 (%)	P@3 (%)	P@3 95% CI	Phys. valid (%)	Attempts	Queries
P+D	86.9	97.0	[95.7, 97.9]	100.0	1.16	584
P+R	86.9	88.1	[85.9, 90.0]	100.0	1.25	540
U+D	66.9	77.0	[74.3, 79.5]	80.0	1.56	910
U+R	66.9	68.1	[65.1, 70.9]	80.0	1.65	866

TABLE IX
Paired contrasts in the weak-oracle regime (Holm over the 8 rows shown).

contrast	metric	b10	b01	RD [95% CI]	p_holm
physics context (feedback on)	Pass@1	200	0	+0.200 [+0.175, +0.225]	1.0×10^{-59}
physics context (feedback on)	Pass@3	200	0	+0.200 [+0.176, +0.226]	1.0×10^{-59}
physics context (feedback off)	Pass@1	200	0	+0.200 [+0.175, +0.225]	1.0×10^{-59}
physics context (feedback off)	Pass@3	200	0	+0.200 [+0.175, +0.225]	1.0×10^{-59}
diagnostic feedback (physics on)	Pass@1	0	0	+0.000 [-0.003, +0.003]	1
diagnostic feedback (physics on)	Pass@3	90	1	+0.089 [+0.072, +0.108]	3.0×10^{-25}
diagnostic feedback (physics off)	Pass@1	0	0	+0.000 [-0.001, +0.001]	1
diagnostic feedback (physics off)	Pass@3	90	1	+0.089 [+0.071, +0.107]	3.0×10^{-25}

TABLE X
Pass@3 (%) as a function of the per-repair oracle query budget (suite size 4). P/U = physics context on/off; D/R = diagnostic feedback / bare rejection; Static = no oracle; P+D/E = P+D with the extended edit neighbourhood; P+D/Q = P+D with Qiskit routing; Qiskit On = transpiler only at optimisation level n.

budget	P+D	U+R
50	77.7	57.7
100	85.1	65.1
200	88.3	68.3
400	91.4	71.4
800	93.7	73.7
1600	95.6	75.6
3200	96.7	76.7

IX. Beyond Unitary Circuits and the Chip Model

The limitations of the main experiments (Section X) invite two follow-ups that we could carry out: programs with measurement, reset, classical control and loops, and a more realistic chip estimate. Both are additions to the released code, not replacements of the results above.

A. Dynamic programs (benchmark v3)

We extended the semantics to the subset of OpenQASM 3 consisting of standard gates, measure, reset, single-bit if/else and constant-body for loops. The classical output distribution of a program is computed exactly by branching on measurement outcomes. It matches Qiskit Aer on teleportation, a reset-based parity check and a loop program to within shot noise (maximum TVD between 0.003 and 0.004 at 2×10^5 shots). Acceptance requires matching register sizes, adjacency of every two-qubit gate (also inside blocks) and equality of the output distribution (TVD $\leq 10^{-6}$) on 16 held-out inputs. The search signal uses a 4-input suite with the same query accounting and counterexample protocol as before.

The edit grammar gains operators for these programs: re-pointing a measurement, coordinated swaps of two measurements (a swap cannot be undone one edit at a time, because the intermediate program scores worse), flipping or re-targeting a classical condition, deleting or inserting a reset, changing a loop count, inserting a classical correction, and a one-dimensional angle search (a loop repeats a rotation, so the score is multi-modal in its angle).

Benchmark v3 has 600 programs from four families (GHZ with terminal measurement, teleportation, a parity check that reuses one ancilla after a reset, a loop) and six fault classes of 100 programs: measurement wiring, control flow, reset, loop, single gate faults and programs with two or three simultaneous faults. Every fault is verified observable. Table XII and Table XIII give the results.

Guided search repairs 92.3% of the programs ($[89.9, 94.2]$) against 48.5% for random mutation hill climbing over the same grammar with the same oracle budget, a paired difference of +0.438 ($[0.396, 0.478]$; 268 programs repaired by guided search only, 5 by random search only; Holm $p < 10^{-70}$). Submitting the faulty program unchanged is accepted for none, so every fault is real. Counterexample feedback makes no measurable difference here (1 against 0 discordant programs). Multi-fault programs are the hardest class (67%): the greedy search needs each edit to raise the score, which two or three interacting faults do not always allow.

Unitary-blind faults. 52.7% of the faults (all wiring, control-flow and reset faults and 16% of the multi-fault programs) leave the unitary of the gate sequence unchanged. This is true by definition, since measurement, reset and conditionals are not part of a unitary, and it means the predicate of the main experiments could not have scored them. Guided search repairs 98.4% of these 316 programs. The remaining limits are that dynamic

TABLE XI
Oracle cost and overfitting of the search signal (searches that used the oracle). Gap = suite agreement minus exact process fidelity of the final candidate.

System	Mean queries	p95 queries	Counterex.	Gap>0.01 (%)	Time (s)
P+D	553	2347	0.07	2.3	0.5
P+R	555	2614	0.00	2.8	0.45
U+D	963	4601	0.07	4.5	0.57
U+R	965	4601	0.00	5.0	0.57
P+D/E	546	2366	0.02	0.9	0.69
P+D/Q	553	2347	0.07	2.3	0.66
Qiskit O1	0	0	0.00	–	0.28
Qiskit O0	0	0	0.00	–	0.34
Qiskit O2	0	0	0.00	–	0.35
Qiskit O3	0	0	0.00	–	0.34

TABLE XII
Benchmark v3 (600 dynamic programs), three attempts, oracle budget 3000. CI: Wilson 95%.

system	n	P@1 (%)	P@3 (%)	P@3 95% CI	Queries
guided + feedback	600	92.2	92.3	[89.9, 94.2]	257
guided, bare rejection	600	92.2	92.2	[89.7, 94.1]	239
random search	600	48.5	48.5	[44.5, 52.5]	195
unchanged program	600	0.0	0.0	[0.0, 0.6]	0

TABLE XIII
Benchmark v3: Pass@3 (%) by fault class (100 programs per class).

fault class	guided+fb	guided	random	unchanged
control_flow	100.0	100.0	70.0	0.0
gate_fault	96.0	96.0	51.0	0.0
loop_fault	91.0	91.0	47.0	0.0
multi_fault	67.0	66.0	29.0	0.0
reset_fault	100.0	100.0	19.0	0.0
wiring	100.0	100.0	75.0	0.0

programs have no noise stage and no minimality check, loops have constant bodies, there is no while loop or multi-bit condition, and the faults are again edits whose inverse lies in the grammar.

B. A calibration-based chip estimate

Adjacency under the identity layout is a weak notion of "runs on the chip". We therefore compile circuits to FakeSherbrooke's native gates with either the identity layout or a compiler-chosen layout and estimate the success probability (ESP) from the snapshot's per-gate and readout errors, together with the duration relative to $\min(T_1, T_2)$. On 200 benchmark references the compiler layout has the higher ESP in every case (Table XIV). Under the identity layout the median ESP is 0.34 for QFT and 0.22 for QAOA, against 0.76 and 0.79 with the compiler layout, and the median QFT duration is 0.47 of $\min(T_1, T_2)$. Two consequences follow. The acceptance predicate's "physically valid" is much looser than "would run well", and the agent's swap-in/swap-back routing, which preserves the unitary exactly, is expensive on hardware. This is an estimate for one calibration snapshot of a simulated device; no circuit was executed on a quantum computer.

C. Language models as the only repair engine

The repository carried a language-model path that none of the numbers above used. We ran it with four models as the only repair engine: no oracle queries, no search and no heuristic fallback, so a reply that does not parse leaves the circuit unchanged. The models are qwen2.5-coder:1.5b (served by Ollama on a laptop CPU) and three models from the free tier of the Gemini API (gemini-3.1-flash-lite, gemini-3.5-flash-lite, gemini-3.5-flash); temperature is 0. All get the same three attempts, the same stage-named feedback and the same acceptance predicate. A run with an API error is not recorded, so service outages do not count as model failures. Because the free tier allows few requests per day, the gemini-3.5-flash run was completed over several days with the resumable runner; model and temperature are the same throughout. We used a stratified subset of 60 circuits (three per fault class and circuit family) and compare on exactly those circuits (Table XV). Runs that stopped early because of the free-tier daily quota or server errors cover only the lowest circuit identifiers, which are the easiest classes, and are excluded from every table.

The local model repairs 16.7% of the circuits within three attempts ($[9.3, 28.0]$), the two small Gemini models 43.3% ($[31.6, 55.9]$) and 51.7% ($[39.3, 63.8]$), and the larger gemini-3.5-flash 86.7% ($[75.8, 93.1]$; 75.0% at the first attempt). Guided search repairs 98.3% of the same circuits. The paired difference is +0.467 ($[0.334, 0.589]$) against the better small model, with 28 circuits repaired by guided search only and none by the model only (Holm $p \approx 5 \times 10^{-8}$), and +0.817 against the local model, but only +0.117 ($[0.038, 0.221]$) against gemini-3.5-flash: 7 circuits repaired by guided search only, none by the model only, exact $p = 0.016$, Holm $p = 0.0625$. With $n = 60$ that gap is not significant after correction and the two

TABLE XIV
Calibration-based estimate on 200 reference circuits (medians): estimated success probability (ESP), native two-qubit gates and duration/min(T1,T2), for the identity layout assumed by the benchmark and a compiler-chosen layout (FakeSherbrooke snapshot).

family	ESP id.	ESP auto	2q id.	2q auto	T ratio id.	T ratio auto
ghz	0.895	0.956	3.5	3	0.025	0.046
qaoa_maxcut	0.223	0.794	36	26	0.515	0.524
qft	0.342	0.76	110	31	0.465	0.169
vqe_h2	0.88	0.938	6	6	0.033	0.068

TABLE XV
LLM-only runs and reference arms on the same 60 circuits (stratified by fault class and family), three attempts, no oracle or search for the LLMs. CI: Wilson 95%.

system	n	P@1 (%)	P@3 (%)	P@3 95% CI
qwen2.5-coder:1.5b	60	11.7	16.7	[9.3, 28.0]
gemini-3.1-flash-lite	60	40.0	43.3	[31.6, 55.9]
gemini-3.5-flash-lite	60	51.7	51.7	[39.3, 63.8]
gemini-3.5-flash	60	75.0	86.7	[75.8, 93.1]
guided search + oracle	60	98.3	98.3	[91.1, 99.7]
static repairs, no oracle	60	60.0	60.0	[47.4, 71.4]
no hardware knowledge	60	78.3	78.3	[66.4, 86.9]
Qiskit compiler (O1)	60	30.0	30.0	[19.9, 42.5]
PyZX	60	5.0	5.0	[1.7, 13.7]

TABLE XVI
Language model as controller of verified tools, on the same 60 circuits. "no model" applies the same tools greedily; "plain LLM" writes the whole circuit. CI: Wilson 95%.

arm	n	P@1 (%)	P@3 (%)	P@3 95% CI
tools: gemini-3.1-flash-lite	60	71.7	95.0	[86.3, 98.3]
plain LLM: gemini-3.1-flash-lite	60	40.0	43.3	[31.6, 55.9]
tools: gemini-3.5-flash-lite	60	81.7	90.0	[79.9, 95.3]
plain LLM: gemini-3.5-flash-lite	60	51.7	51.7	[39.3, 63.8]
tools: no model (greedy)	60	98.3	98.3	[91.1, 99.7]

intervals overlap. Static repairs without any oracle reach 60.0%; their advantage over the two small Gemini models (+0.167 and +0.083) is not significant after correction, but they are clearly worse than gemini-3.5-flash (-0.267, $[-0.391, -0.133]$; Holm $p = 0.002$). The two small Gemini models do not differ significantly ($+0.083$, $[-0.040, 0.202]$); gemini-3.5-flash is clearly better than both (+0.350 and +0.433, with 21 and 26 circuits against none; Holm $p < 10^{-5}$). By class, the small Gemini models repair 67–92% of syntax and 75–83% of gate-redundancy circuits but only 8–17% of semantic-deviation circuits and 17–33% of hardware faults, whereas gemini-3.5-flash repairs 100% of the first two, 75% of semantic-deviation and of structural circuits, and 83% of hardware faults (guided search: 100% in every class except structural, 92%). The small models' failures end at the semantic stage (16 and 17 circuits) or the connectivity stage (16 and 11); those of gemini-3.5-flash at the semantic stage (7) or the connectivity stage (1). The small Gemini models always returned a parsable circuit, gemini-3.5-flash did not for 5.0% of the circuits and the local model for 26.7%. A circuit cost 1.4–2.7 calls, 1,650–5,568 tokens and 12–78 seconds.

These results are not verdicts on language models. The larger Gemini model repairs most circuits with no oracle at all, so on this benchmark an oracle-guided search is not needed to reach a high rate, only for the last points (86.7% against 98.3%, a difference that is not significant at $n = 60$) and for the guarantee that every submitted circuit has been verified. None of the four models is a frontier model; the frontier models used in other work report much higher rates with a single retry [3] on a different benchmark and protocol; gemini-3.8-flash did not finish on the free tier; with 60 circuits and a single run our intervals are wide; the circuits are generated faults; and we did not tune prompts beyond the physical-constraint text already in the repository. What the runs show is that the harness hosts language models under the same predicate and budgets and records their token and time cost, that the three small models do not substitute for the oracle-guided search here, and that a larger one comes close.

D. A language model as the controller of verified tools

A second way to use a language model is to let it call tools instead of writing the circuit. In the llm_tools arm the model emits one JSON tool call per turn: diagnose, lint, localise, list the best oracle-scored single-instruction edits, apply one, run the static repairs, replace the circuit, or submit it to the acceptance predicate (at most three submissions and 14 calls). Information and budgets are those of the deterministic agent: no reference, counted oracle queries, stage-named feedback. The ablation is the same tool set driven by a fixed greedy policy with no model at all (tools_greedy). Table XVI gives the result on the same 60 circuits.

With the tools, the two Gemini models repair 95.0% ($[86.3, 98.3]$) and 90.0% ($[79.9, 95.3]$) of the circuits, against 43.3% and 51.7% when they write the circuit (paired differences +0.517 and +0.383; 31 and 26 circuits only with tools, 0 and 3 only without; Holm $p < 10^{-4}$). But the same tools with no model repair 98.3%. The model-driven arms are not significantly worse (-0.033, $[-0.122, 0.047]$ and -0.083, $[-0.180, -0.012]$; Holm $p = 0.63$ and 0.13): one circuit was repaired only by the model-driven arm and three only by the baseline for the first model, none and five for the second. They cost 5.7–7.0 calls and 9.9–15.2 thousand tokens per circuit against about two calls and 2 thousand tokens for the plain arm, and miss hardware faults (83%). The reading is that correctness comes from the oracle-scored edit suggestions and the verifier, not from the model's knowledge of circuits: a

model can drive the tools, which lifts it far, but it adds nothing on top of them. This does not show that language models repair circuits; a stronger result would be a model that uses the tools and beats the greedy baseline, and none of the three models we tried (the two above and a local 1.5-billion-parameter model that did not finish) does. A first run of this arm was discarded because the greedy policy kept state between circuits and scored 61.7%; the state is now reset per run and covered by a test. The tool arm was run only with the two small Gemini models; the larger gemini-3.5-flash (86.7% without tools) was not run in it, so the comparison with the no-model baseline is untested for a stronger model.

E. A post-hoc change to the agent

Twelve circuits of the benchmark remained unrepaired by the extended agent, seven of them QAOA circuits with a misplaced rotation. In such a circuit the two CX gates around the rotation have nothing between them on their wires, so they cancel exactly; the static diagnosis calls the circuit redundant and the cancellation step removes them, after which three edits instead of one are needed. Applying the cancellation only to circuits that the oracle already scores as functionally correct (the guard arm) repairs nine more circuits (the seven misplaced rotations and two QFT circuits with a removed gate) and none fewer (98.8% $\rightarrow$ 99.7%, $[99.1, 99.9]$; exact McNemar $p = 0.004$ unadjusted; mean oracle queries 547 $\rightarrow$ 489). This change was designed after reading these failures on the benchmark, so 99.7% is a development-set number, not an independent test, and the main tables are unchanged. Three circuits still fail: one 8-qubit QAOA circuit with a removed rotation and two QFT circuits with two simultaneous angle errors.

F. A pilot on real bugs

Bugs4Q collects real bugs of Qiskit programs. We ran the Python scripts of its entries in a sandbox with a compatibility shim and kept the entries whose buggy and fixed scripts build circuits that differ in logic on the same qubits. Reading the differences by hand, five of the entries could be turned into (faulty, reference) pairs; most entries are library, API or version bugs without a circuit-level fix. Three pairs use the full acceptance predicate, two (whose own reference contains non-adjacent two-qubit gates) use logic-only acceptance. Guided repair fixes three of the five (random search with the same grammar and budget: two); the two it misses are a QFT whose qubit-order swap is missing and four missing Hadamard gates. The language models fix none of the five from the faulty circuit alone and one (an H gate where an X was intended) when they are also told the program's output on the all-zero input. The arms do not receive the same information, and five pairs support no rate and no ranking; the pilot shows only that the pipeline runs on bugs that people wrote, and where it fails.

X. Threats to Validity and Limitations

Edit-inversion benchmark. Faults are single- or double-instruction edits and the neighbourhood contains their inverses. The 97% rate therefore characterises the agent on this fault distribution and says little about unrestricted faults. The informative evidence is the per-mode table, the weak-oracle regime and the budget sweep.

Language and fault coverage. The acceptance predicate of the main experiments compares unitaries, device admissibility and noisy output distributions, so it covers the purely unitary subset of OpenQASM 3 [24]. Section IX adds an exact semantics for measurement, reset, single-bit conditionals and constant-body loops, but not while loops, multi-bit conditions, timing beyond delay, or a noise stage for such programs. Empirical studies of real quantum bugs report that a large share concern API usage, platform and cross-layer behaviour rather than gate-level edits [25], [26]. Benchmarks v2 and v3 therefore do not represent field prevalence: v3 widens the fault types (wiring, classical control, reset, loops, multi-fault) but its faults are still injected edits whose inverse lies in the search grammar.

Routing by construction. The agent's default router equals the construction used to build physical-fault references. We tested this directly by swapping in Qiskit's SABRE router with an exact final-layout undo: acceptance is unchanged (97.0%, zero discordant circuits), and the routed circuits are 1.1% smaller than the reference on average, so agreement is not an artefact of sharing a router. The physical-knowledge contrast remains a capability contrast (Section VIII).

Oracle strength. Agreement on a handful of input states is a weaker specification than a full test suite, and the oracle returns scores of many candidate edits, which is substantial information about the reference. We bound this by counting every edit as a query and by measuring the overfitting gap; the budget and suite size are experimental variables, not hidden constants.

Physical model. Connectivity is checked on virtual indices under the identity layout; the noise model is depolarising with analytic readout error; FakeSherbrooke is a model, not a live device with calibration drift. A routing-aware or transpiled evaluation would be strictly more demanding; initial placement, routing and calibration-aware mapping change success rates substantially on superconducting hardware [27], [28], so identity-layout validity is a necessary rather than sufficient condition. Exact unitary comparison does not scale: the validator therefore also offers a sampled average-fidelity check (and skips the density-matrix noise stage) above ten qubits, which gives one-sided statistical evidence only, and structure-aware verification such as automata-based [29] or ZX-based [30] equivalence checking would be the principled replacement. All reported results use the exact check (≤ 8 qubits).

Missing baselines. The language-model evidence is three small models on 60 circuits; no frontier model was run to completion, so nothing here characterises language-model repair in general. HornBro (FSE 2025) has public code; it

needs Linux and its own Python 3.10 environment, and an adapter and a Colab notebook that apply our predicate to its output exist, but no HornBro result is reported yet. We report no QBugLM or UnitAR run: a search on 2026-10-22 found no public code repository for either. No circuit was executed on real quantum hardware.

Post-hoc changes and training data. Two arms were designed after reading failures on the benchmark: the extended edit neighbourhood and the guarded cancellation. Their numbers describe a development set; only the arms fixed before the first run (the main table) are untouched by this. A fresh-seed training set (1900 pairs, none shared with the benchmark) is provided for future fine-tuning, but 21% of its reference circuits coincide with benchmark references because the circuit families are deterministic, so results of a model trained on it must say so.

Statistics. The 1000 circuits are not independent draws of real faults; they are a generated population, so intervals describe sampling over that population. Three circuit families and a fixed fault taxonomy limit generalisation.

XI. Conclusion

Reported success rates for quantum-circuit repair depend on conditions that are easy to violate silently: an information boundary around the reference, a benchmark whose references satisfy the predicate, and faults the predicate can detect. An audit found all three broken in an existing physics-constrained repair benchmark, and a corrected protocol and benchmark remove the defects while making the remaining limitations measurable. On the corrected benchmark a repair agent with analytic angle solving and oracle-guided search is accepted on 97.0% of circuits at a matched budget of three attempts; the contribution of physical knowledge is confined to the physical fault class, oracle access is what separates semantic repair from static repair, and diagnostic feedback matters when the oracle is weak. Extending the semantics to measurement, reset and classical control lets the protocol score faults a unitary check cannot see, and a calibration-based estimate shows how much looser "valid on the chip" is than "runs well". The natural next steps are frontier language models and the published repair tools under the same predicate, hard faults outside the edit neighbourhood, and execution on real devices.

Acknowledgment

The author thanks the open-source quantum software community.

References

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," Quantum, vol. 2, p. 79, 2018.

[2] X. Guo, J. Zhao, and P. Zhao, "On repairing quantum programs using ChatGPT," in Proc. IEEE/ACM 5th Int. Workshop on Quantum Software Engineering (Q-SE), 2024, pp. 9–16.

[3] A. B. B. Pham, H. T. Nguyen, and M. Usman, "QBugLM: An agentic benchmarking framework for LLM-based quantum software debugging," arXiv preprint arXiv:2606.07314, 2026, also IEEE Int. Conf. Quantum Software (QSW) 2026, pp. 185–191, doi:10.1109/qsw72780.2026.00028.

[4] R. Duncan, A. Kissinger, S. Perdrix, and J. van de Wetering, "Graph-theoretic simplification of quantum circuits with the ZX-calculus," Quantum, vol. 4, p. 279, 2020.

[5] A. Kissinger and J. van de Wetering, "PyZX: Large scale automated diagrammatic reasoning," in Proc. 16th Int. Conf. Quantum Physics and Logic (QPL), EPTCS 318, 2020.

[6] Z. Li, J. Peng, Y. Mei, S. Lin, Y. Wu, O. Padon, and Z. Jia, "Quarl: A learning-based quantum circuit optimizer," Proc. ACM Program. Lang., vol. 8, no. OOPSLA1, 2024.

[7] S. Tan, L. Lu, D. Xiang, T. Chu, C. Lang, J. Chen, X. Hu, and J. Yin, "HornBro: Homotopy-like method for automated quantum program repair," Proceedings of the ACM on Software Engineering, vol. 2, pp. 734–756, 2025.

[8] Y. Li, H. Pei, L. Huang, B. Yin, and K.-Y. Cai, "Automatic repair of quantum programs via unitary operation," ACM Transactions on Software Engineering and Methodology, vol. 33, 2024.

[9] P. Zhao, Z. Miao, S. Lan, and J. Zhao, "Bugs4Q: A benchmark of existing bugs to enable controlled testing and debugging studies for quantum programs," Journal of Systems and Software, vol. 205, p. 111805, 2023.

[10] H. Ohto, Y. Ishimoto, S. Matsumoto, and S. Kusumoto, "On the reproducibility of quantum software defect datasets: A case study of Bugs4Q," arXiv preprint arXiv:2606.27124, 2026.

[11] N. C. L. Ramalho, H. A. de Souza, and M. L. Chaim, "Testing and debugging quantum programs: The road to 2030," ACM Transactions on Software Engineering and Methodology, vol. 34, 2024.

[12] N. Oldfield, C. Laaber, T. Yue, and S. Ali, "Faster and better quantum software testing through specification reduction and projective measurements," ACM Transactions on Software Engineering and Methodology, vol. 34, 2024.

[13] D. Rovara, L. Burgholzer, and R. Wille, "A framework for debugging quantum programs," in Proc. IEEE Int. Conf. Quantum Software (QSW), 2025, arXiv:2412.12269.

[14] G. Li, L. Zhou, N. Yu, Y. Ding, M. Ying, and Y. Xie, "Projection-based runtime assertions for testing and debugging quantum programs," Proceedings of the ACM on Programming Languages, vol. 4, 2020.

[15] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, "GenProg: A generic method for automatic software repair," IEEE Transactions on Software Engineering, vol. 38, no. 1, pp. 54–72, 2012.

[16] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in Proc. 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE), 2015, pp. 532–543.

[17] A. Solar-Lezama, L. Tancau, R. Bodík, S. A. Seshia, and V. Saraswat, "Combinatorial sketching for finite programs," in Proc. 12th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2006.

[18] K. M. Nakanishi, K. Fujii, and S. Todo, "Sequential minimal optimization for quantum-classical hybrid algorithms," Physical Review Research, vol. 2, p. 043158, 2020.

[19] M. Ostaszewski, E. Grant, and M. Benedetti, "Structure optimization for parameterized quantum circuits," Quantum, vol. 5, p. 391, 2021.

[20] IBM Quantum, "Build noise models," IBM Quantum Documentation, 2026, accessed: Sep. 13, 2026. [Online]. Available: https://quantum.cloud.ibm.com/docs/en/guides/build-noise-models.

[21] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153–157, 1947.

[22] R. G. Newcombe, "Improved confidence intervals for the difference between binomial proportions based on paired data," Statistics in Medicine, vol. 17, no. 22, pp. 2635–2650, 1998.

[23] S. Holm, "A simple sequentially rejective multiple test procedure," Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65–70, 1979.

[24] A. Cross, A. Javadi-Abhari, T. Alexander, N. de Beaudrap, L. S. Bishop, S. Heidel, C. A. Ryan, P. Sivarajah, J. Smolin, J. M. Gambetta, and B. R. Johnson, "OpenQASM 3: A broader and deeper quantum assembly language," ACM Transactions on Quantum Computing, vol. 3, no. 3, pp. 1–50, 2022.

[25] J. Luo, P. Zhao, Z. Miao, S. Lan, and J. Zhao, "A comprehensive study of bug fixes in quantum programs," in Proc. IEEE

Int. Conf. Software Analysis, Evolution and Reengineering (SANER), 2022.
[26] M. Paltenghi and M. Pradel, "Bugs in quantum computing platforms: An empirical study," Proceedings of the ACM on Programming Languages, vol. 6, 2021.
[27] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, and M. Martonosi, "Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers," in Proc. 24th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019.
[28] P. Nation, A. A. Saki, S. Brandhofer, L. Bello, S. Garion, M. Treinish, and A. Javadi-Abhari, "Benchmarking the performance of quantum computing software for quantum circuit creation, manipulation and compilation," Nature Computational Science, vol. 5, pp. 427–435, 2025.
[29] Y.-F. Chen, K.-M. Chung, O. Lengál, J.-A. Lin, W.-L. Tsai, and D.-D. Yen, "An automata-based framework for verification and bug hunting in quantum circuits," Proceedings of the ACM on Programming Languages, vol. 7, pp. 1218–1243, 2023.
[30] T. Peham, L. Burgholzer, and R. Wille, "Equivalence checking of quantum circuits with the ZX-calculus," IEEE Journal on Emerging and Selected Topics in Circuits and Systems, vol. 12, pp. 662–675, 2022.